

Pattern Matching in C# 8 with Examples

In this article, I am going to discuss **Pattern Matching Enhancements in C#** with Examples. Please read our previous article where we discussed **Default Interface Methods in C#** with Examples. Pattern Matching is one of the powerful features in C# and in C# 8, it is further enhanced. Pattern Matching in C# is a mechanism that tests a value i.e. whether the value has a specific shape or not. If the value is in a specific shape, then it will extract the data from the value.

What is Pattern Matching in C#?

Pattern matching is the process of taking an expression and testing whether it matches certain criteria, such as “being a specified type” or “matching a specified constant value”.

Enhanced Pattern Matching in C# 8

Pattern Matching was introduced as part of C# 7, but after a while, the .Net Development team has found that they need more time to finish this feature. For this reason, the .Net Development team divided Pattern Matching into two parts. The Basic **Pattern Matching is already delivered with C# 7** and the Advanced Patterns Matching is delivered with C# 8.

C# 7.0 first introduced syntax for type patterns and constant patterns by using the `is` expression and the `switch` statement. These features represented the first tentative steps toward supporting programming paradigms where data and functionality live apart. As the industry moves toward more microservices and other cloud-based architectures, other language tools are needed.

C# 8.0 expands this vocabulary so you can use more pattern expressions in more places in your code. Consider these features when your data and functionality are separate. Consider pattern matching when your algorithms depend on a fact other than the runtime type of an object. These techniques provide another way to express designs.

In addition to new patterns in new places, C# 8.0 adds recursive patterns. Recursive patterns are patterns that can contain other patterns. In C# 8, the .Net Development team has introduced the following Patterns.

1. **Switch Expressions**
2. **Property Patterns**
3. **Tuple Patterns**
4. **Positional Patterns**

The main benefit of Pattern Matching is that it provides a flexible and powerful way of testing data against a series of conditions (like if-else) and performing some computations based on the condition met.

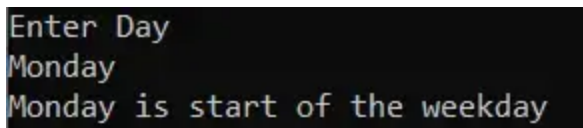
Switch Expressions in C#

We are already familiar with the original `switch` statement in C#, which returns one value. The `switch` expression enables us to use more concise expression syntax which returns an expression instead of a value. The `switch` expressions syntax is a little bit different compared to the old `switch` statement. There are fewer repetitive `case` and `break` keywords and fewer curly braces. `Switch` expression allows us to use Pattern Matching within the `switch` scope.

Let us understand switch expressions with some examples. Please have a look at the below code. This is a simple switch case example. Here, we are taking the day from the user, and then using a switch case statement we are setting the message based on the day.

```
using System;
namespace Csharp8Features
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Enter Day");
            string day = Console.ReadLine();
            string message = string.Empty;
            switch (day.ToUpper())
            {
                case "SUNDAY":
                    message = "Weekend";
                    break;
                case "MONDAY":
                    message = "start of the weekday";
                    break;
                case "FRIDAY":
                    message = "end of the weekday";
                    break;
                default:
                    message = "Invalid day";
                    break;
            }
            Console.WriteLine($"{day} is {message}");
        }
    }
}
```

Output:

A screenshot of a terminal window with a black background and light blue text. It shows the program's execution: the prompt "Enter Day", the user input "Monday", and the resulting output "Monday is start of the weekday".

```
Enter Day
Monday
Monday is start of the weekday
```

The above syntax is fine up to C# 7. But from C# 8, we can write the above example using the switch expression. Let us see how to use the switch expressions to rewrite the previous example.

```
using System;
namespace Csharp8Features
{
    class Program
    {
        static void Main(string[] args)
        {
```

```

        Console.WriteLine("Enter Day");
        string day = Console.ReadLine();

        var message = day.ToUpper() switch
        {
            "SUNDAY" => "Weekend",
            "MONDAY" => "start of the weekday",
            "FRIDAY" => "end of the weekday",
            _ => "Invalid day"
        };
        Console.WriteLine($"{day} is {message}");
    }
}

```

The above example exactly gives you the same output as the previous example. When you run the above code, you will get the following output.

```

Enter Day
Monday
Monday is start of the weekday

```

Understanding the Switch Expression in C#:

Let us compare the syntax of both switch case and switch expression and understand how to switch expression formed.

Switch Case statement	Switch Expression statement
<pre> switch (day.ToUpper()) { case "SUNDAY": message = "Weekend"; break; case "MONDAY": message = "start of the weekday"; break; case "FRIDAY": message = "end of the weekday"; break; default: message = "Invalid day"; break; } </pre>	<pre> var message = day.ToUpper() switch { "SUNDAY" => "Weekend", "MONDAY" => "start of the weekday", "FRIDAY" => "end of the weekday", _ => "Invalid day" }; </pre> Here, 1. We move switch expression before the switch statement 2. Remove the case and break statement. 3. Using goes to operator (=>) 4. _=> (underscore goes to) is used for default block

Here,

1. We move the switch expression before the switch statement
2. Remove the case and break statements.

3. Using goes to operator (=>) to assign to select the value. Before the goes to operator, we have input and after the goes to operator, we have the expression.
4. `_=>` (underscore goes to) is used for default block

Note: Comparing the switch case, the switch expression is easy and lesser number of codes.

Real-time Example to Understand Switch Expression in C#:

Let us understand Switch Expression with another but complex example. The following example demonstrates a Switch Expression in C#, which converts values of an enum representing visual directions in an online map to the corresponding cardinal directions.

Let us first create the following Direction enum. This enum contains four constant values i.e. Up, Down, Right, and Left

```
public enum Direction
{
    Up,
    Down,
    Right,
    Left
}
```

Next, create the following Orientation enum. This enum contains four constant values i.e. North, South, East, and West

```
public enum Orientation
{
    North,
    South,
    East,
    West
}
```

Now, let us first see how to get the cardinal directions using the switch case statement, and then we will see how to convert the same example to use the Switch Expression.

Example using Switch Case Statement in C#:

```
using System;
namespace Csharp8Features
{
    public enum Direction
    {
        Up,
        Down,
        Right,
        Left
    }
}
```

```

public enum Orientation
{
    North,
    South,
    East,
    West
}
class Program
{
    static void Main(string[] args)
    {
        var direction = Direction.Left;
        Console.WriteLine($"Map view direction is {direction}");
        Orientation? orientation;
        switch (direction)
        {
            case Direction.Up:
                orientation = Orientation.North;
                break;
            case Direction.Down:
                orientation = Orientation.South;
                break;
            case Direction.Left:
                orientation = Orientation.West;
                break;
            case Direction.Right:
                orientation = Orientation.East;
                break;
            default:
                throw new ArgumentOutOfRangeException(nameof(direction), $
expected direction value: {direction}");
        }
        Console.WriteLine($"Cardinal Orientation is {orientation}");
    }
}

```

Output:

```

Map view direction is Left
Cardinal orientation is West

```

Example using Switch Expression in C#:

```

using System;
namespace Csharp8Features
{
    public enum Direction
    {

```

```

        Up,
        Down,
        Right,
        Left
    }
    public enum Orientation
    {
        North,
        South,
        East,
        West
    }
    class Program
    {
        static void Main(string[] args)
        {
            var direction = Direction.Left;
            Console.WriteLine($"Map view direction is {direction}");
            Orientation? orientation = direction switch
            {
                Direction.Up => Orientation.North,
                Direction.Down => Orientation.South,
                Direction.Left => Orientation.West,
                Direction.Right => Orientation.East,
                _ => throw new ArgumentOutOfRangeException(nameof(direction),
expected direction value: {direction}),
            };
            Console.WriteLine($"Cardinal Orientation is {orientation}");
        }
    }
}

```

Output:

```

Map view direction is Left
Cardinal orientation is West

```

Property Patterns in C# 8

The Property Pattern in C# is used for checking and comparing the values of properties. That means the property pattern tests whether an expression's properties/fields match the values of specified properties/fields. Each corresponding property or field must match and the expression must not be null. Let us understand this with an example.

In the below example we use Property Pattern which inspects the Length and Height properties of a Rectangle object. The property pattern begins with checking whether the input is a specified type. Once the property pattern ensures that the input is a Rectangle, then only it will inspect the Length property and the Height property of the input. The pattern will match the expression if the input is a Rectangle with a Length of 5 and a height of 10.

```

using System;
namespace Csharp8Features
{
    class Program
    {
        static void Main(string[] args)
        {
            Rectangle rectangle = new Rectangle {Length = 5, Height = 10};
            IsSpecificRectanglee(rectangle);
        }
        public static void IsSpecificRectanglee(Rectangle rectangle)
        {
            if (rectangle is Rectangle { Length: 5, Height: 10 } specificRectangle)
            {
                Console.WriteLine($"Rectangle Length: {specificRectangle.Length} and Height: {specificRectangle.Height}");
            }
        }
    }
    public class Rectangle
    {
        public double Length { get; set; }
        public double Height { get; set; }
    }
}

```

Output:

In the above example, the following piece of code exactly does the same. First, it will check whether the input i.e. rectangle is Rectangle and if so then it will check the property Length and Height values. If Length = 5 and Height = 10, then only it will go inside the if block and execute the statement.

```

if (rectangle is Rectangle {Length: 5, Height: 10})
{
    Console.WriteLine($"Rectangle Length: {rectangle.Length} and Height: {rectangle.Height}");
}

```

Now, let us see how we can achieve the same prior to C# 8. Please have a look at the below example.

```

using System;
namespace Csharp8Features
{
    class Program
    {
        static void Main(string[] args)
        {

```

```

        Rectangle rectangle = new Rectangle { Length = 5, Height = 10 };
        IsSpecificRectanglee(rectangle);
    }
    public static void IsSpecificRectanglee(Rectangle rectangle)
    {
        //Older Version
        //if (rectangle.GetType() == typeof(Rectangle))
        //{
        //    var rect = (Rectangle)rectangle;
        //    if (rect.Length == 5 && rect.Height == 10)
        //    {
        //        Console.WriteLine($"Rectangle Length: {rectangle.Length}
Height: {rectangle.Height}");
        //    }
        //}
        //Older Version
        var rect = rectangle as Rectangle;
        if (rect != null)
        {
            if (rect.Length == 5 && rect.Height == 10)
            {
                Console.WriteLine($"Rectangle Length: {rectangle.Length} :
{rectangle.Height}");
            }
        }
    }
}
public class Rectangle
{
    public double Length { get; set; }
    public double Height { get; set; }
}
}

```

Now, you can compare the pattern matching code prior to C# 7 and look at how the C# 8 code is more clear. The new style removes the redundant code and the typecasting or the ugly operators like `typeof` or `as`.

Real-Time Example of Property Pattern in C#

Let us consider an eCommerce site that computes sales tax based on the buyer's Address i.e. based on state. The sales tax computation isn't the core responsibility of an Address class. It will change over time, likely more often than address format changes. The amount of sales tax depends on the State property of the address. The following example uses the property pattern to compute the sales tax from the address and the price. In the below example, we use switch expression along with Property Pattern in C#.

```

using System;
namespace Csharp8Features
{
    class Program
    {

```

```

static void Main(string[] args)
{
    double salePrice = 100;
    Address address = new Address { State = "DELHI" };
    var salesTax = ComputeSalesTax(address, salePrice);
    Console.WriteLine($"State: {address.State}, salePrice: {salePrice}
salesTax: {salesTax}");
}

public static double ComputeSalesTax(Address location, double salePrice)
{
    var salesTax = location switch
    {
        { State: "DELHI" } => salePrice * 0.06,
        { State: "MUMBAI" } => salePrice * 0.075,
        { State: "PUNE" } => salePrice * 0.05,
        _ => 0
    };
    return salesTax;
}
}

public class Address
{
    public string State { get; set; }
}
}

```

Output:

```
State: DELHI, salePrice: 100, and salesTax: 6
```

Tuple Patterns in C# 8

The Tuple Pattern in C# can be used to pattern match multiple input values. We can write the complex logic very clean using the Tuple Pattern. When we have multiple inputs and we need to match by the combination of inputs then Tuple Pattern is useful.

Let us understand C# 8 Tuple Pattern with an example. In the following example, we created a method called SportsLike. This method takes three string parameters or we can say 1 tuple. It compares this tuple (three-string parameters values) with tuple templates defined as switch expression cases, and the expression of the matching tuple value is being returned to the caller. If no case is matched, the default case is executed. Using the Console.WriteLine function, we output the return value of the function.

```

using System;
namespace Csharp8Features
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine(SportsLike("Cricket", "Football", "Swimming"));
        }
    }
}

```

```

        Console.WriteLine(SportsLike("Cricket", "Football", "Racing"));
    }
    static string SportsLike(string sport1, string sport2, string sport3)
    {
        var result = (sport1, sport2, sport3) switch
        {
            //matches if Cricket, Football, and Swimming given as input
            ("Cricket", "Football", "Swimming") => "I like Cricket, Football and Swimming.",
            //matches if Cricket, Football, and Baseball given as input
            ("Cricket", "Football", "Baseball") => "I like Cricket, Football and Baseball.",
            //matches if Hockey, Football, and Swimming given as input
            ("Hockey", "Football", "Swimming") => "I like Hockey, Football and Swimming.",
            //matches if Table Tennis, Football, and Swimming given as input
            ("Table Tennis", "Football", "Swimming") => "I like Table Tennis, Football and Swimming.",
            //Default case
            (_, _, _) => "Invalid input!"
        };
        return result;
    }
}

```

Output:

The output of the program is as follows, the first call matches the first case label and the second call results in the default case as Racing is not part of any case label.

```

I like Cricket, Football, and Swimming.
Invalid input!

```

Realtime Example to Understand Tuple Pattern in C#

We need to write a program that should determine discounts based on the following conditions

1. If the order is paid by Credit Card and the country is India then return 20 to apply 20 percent discounts.
2. If the order is paid by Wire Transfer and the country is the USA then return 15 to apply 15 percent of discount
3. if the order amount is greater than 5000 return 10 to apply a 10 percent discount
4. otherwise return 0 for 0 percentage

```

using System;
namespace Csharp8Features
{
    class Program

```

```

{
    static void Main(string[] args)
    {
        CustomerOrder customerOrder1 = new CustomerOrder()
        {
            PaymentMethod = PaymentMethods.CreditCard,
            Country = "India",
            Amount = 2000
        };
        CustomerOrder customerOrder2 = new CustomerOrder()
        {
            PaymentMethod = PaymentMethods.WireTransfer,
            Country = "USA",
            Amount = 3000
        };
        Console.WriteLine($"Country: {customerOrder2.Country}, Payment Method: {customerOrder1.PaymentMethod}, Order Discount : {GetOrderDiscount(customerOrder1)}");
        Console.WriteLine($"Country: {customerOrder2.Country}, Payment Method: {customerOrder2.PaymentMethod}, Order Discount : {GetOrderDiscount(customerOrder2)}");
    }
    public static int GetOrderDiscount(CustomerOrder order)
    {
        return (order.PaymentMethod, order.Country) switch
        {
            (PaymentMethods.CreditCard, "India") => 20,
            (PaymentMethods.WireTransfer, "USA") => 15,
            (_, _) when order.Amount > 5000 => 10,
            _ => 0 // unknown or Default
        };
    }
}
public class CustomerOrder
{
    public PaymentMethods PaymentMethod { get; set; }
    public string Country { get; set; }
    public double Amount { get; set; }
}
public enum PaymentMethods
{
    CreditCard,
    WireTransfer
}
}

```

Output:

```

Country: USA, Payment Method : CreditCard, Order Discount : 20
Country: USA, Payment Method : WireTransfer, Order Discount : 15

```

Positional Patterns in C# 8

Apart from properties and tuples, patterns can be matched even for positions. Positional pattern matching works with the help of deconstructors introduced in C# 7. Deconstructor allows setting its properties into discrete variables and allows the use of positional patterns in a compact way without having to name properties.

The Positional Pattern in C# 8 is useful when testing a type that can be deconstructed. Deconstruction is a process of unpacking types into parts and storing them into new variables (object deconstruction). For example, a class can be split into its properties or a tuple can be split into its individual items. The Positional Pattern in C# 8 can deconstruct an input expression and then test if the resulting variables match against a pattern specified in parentheses.

Example to Understand Positional Patterns in C# 8

C# provides built-in support for deconstructing the record and tuple types. However, for other types, the compiler requires a Deconstruct method to be implemented. Let us understand this with an example. Let us create a class called Rectangle and include the custom Deconstruct method as shown below. Each value to be deconstructed is referred to by an out parameter. The Deconstruct method splits the Triangle type and returns a length variable and a height variable.

```
public class Rectangle
{
    public double Length { get; set; }
    public double Height { get; set; }
    public void Deconstruct(out double length, out double height)
    {
        length = Length;
        height = Height;
    }
}
```

Please note that here method name Deconstruct is required by the compiler and any other name with the same method body won't work. Now, you can deconstruct an instance of the Rectangle class named rectangle with an assignment as follows:

```
Rectangle rectangle = new Rectangle {Length = 20, Height = 40};
var (length, height) = rectangle;
```

In the following example, the rectangle type is deconstructed and length is tested against the 20 patterns (a constant pattern) and height is tested against _ (the discard pattern). The pattern reflects the position each variable has within the Deconstruct method. So, the first value coming out of the Deconstruct method is the first value we are going to match on and so forth. Discards (_) can be used where any value is accepted at that position. So, the pattern will match if the rectangle has a length of 20 and a height of any value.

```
using System;
namespace Csharp8Features
{
    class Program
    {
        static void Main(string[] args)
```

```

    {
        Rectangle rectangle = new Rectangle { Length = 20, Height = 40 };
        var (length, height) = rectangle;
        Console.WriteLine($"The rectangle Length: {length} and Height: {height}");
        if (rectangle is (20, _) rect)
        {
            Console.WriteLine("The rectangle has a length of 20");
        }
    }
}
public class Rectangle
{
    public double Length { get; set; }
    public double Height { get; set; }
    public void Deconstruct(out double length, out double height)
    {
        length = Length;
        height = Height;
    }
}
}

```

Output:

```

The rectangle Length: 20 and Height: 40
The rectangle has a length of 20

```

Note: This feature is inspired by functional programming and uses a de-constructor. The deconstructor should be a public method named Deconstructor. It has out parameters for values that need to deconstruct.

Realtime Example to Understand Positional Patterns in C# 8

We need to create a program that should determine if a customer is eligible for free shipping. The Criteria to determine free shipping is; if a customer is from the USA. We need to create a deconstructor in Customer and Address classes.

Address Class:

```

public class Address
{
    public int PostalCode { get; set; }
    public string Street { get; set; }
    public string Country { get; set; }
    // Deconstruct for Address
    public void Deconstruct(out int postalCode, out string street, out string country)
    {
        postalCode = PostalCode;
        street = Street;
        country = Country;
    }
}

```

```

    }
}

```

Customer Class:

```

public class Customer
{
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public string Email { get; set; }
    public Address CustomerAddress { get; set; }
    //Deconstruct for Customer
    public void Deconstruct(out string firstname, out string lastname, out string email, out Address address)
    {
        firstname = FirstName;
        lastname = LastName;
        email = Email;
        address = CustomerAddress;
    }
}

```

Now, we will create our function that can match parameters as position; (_,_, "USA"). The function is returning a boolean value if Address's third parameter is "USA".

```

public static bool IsFreeShippingEligible(Customer customer)
{
    // If customer is from the USA then Free shipping applies.
    return customer is Customer(_, _, _, (_, _, "USA"));
}

```

Complete Example Code:

```

using System;
namespace Csharp8Features
{
    class Program
    {
        static void Main(string[] args)
        {
            Customer customer = new Customer()
            {
                FirstName = "Sam",
                LastName = "Taylor",
                Email = "info@dotnettutorials.net",
                CustomerAddress = new Address() { PostalCode = 755019, Street
                "Newyork", Country = "USA"}
            };
            Console.WriteLine($"Is Free Shipping Eligible : {IsFreeShippingEligible(customer)}");
        }
    }
}

```

```

    }
    public static bool IsFreeShippingEligible(Customer customer)
    {
        // If customer is from USA then Free shipping applies.
        return customer is Customer(_, _, _, (_, _, "USA"));
    }
}
public class Customer
{
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public string Email { get; set; }
    public Address CustomerAddress { get; set; }
    //Deconstruct for Customer
    public void Deconstruct(out string firstname, out string lastname, out string email, out Address address)
    {
        firstname = FirstName;
        lastname = LastName;
        email = Email;
        address = CustomerAddress;
    }
}
public class Address
{
    public int PostalCode { get; set; }
    public string Street { get; set; }
    public string Country { get; set; }
    // Deconstruct for Address
    public void Deconstruct(out int postalCode, out string street, out string country)
    {
        postalCode = PostalCode;
        street = Street;
        country = Country;
    }
}
}

```

Output:

```
Is Free Shipping Eligible : True
```

In the next article, I am going to discuss [Using Declarations in C# 8](#) with Examples. Here, in this article, I try to explain **Pattern Matching Enhancements in C# 8** with Examples. I hope you enjoy this Pattern Matching Enhancements in C# 8 with Examples article.